

pwd affiche le répertoire courant

cd R se place dans le répertoire R

cd .. se place dans le répertoire précédent

ls R liste les fichiers du répertoire R

ls -a liste les fichiers cachés avec ls | -R affiche la taille des fichiers de manière lisible (en K, M, G)

ls -l liste des fichiers avec les informations détaillées

1- Type et permissions montre le type (d pour dossier, - pour fichier) et les permissions pour l'utilisateur, groupe, autres (-rw-r--r--).

2- Nombres de liens : Compte les liens symboliques pointant vers ce fichier ou dossier

3- Propriétaire : L'utilisateur qui possède l'élément (user)

4- Groupe : Le groupe propriétaire de l'élément

5- Taille : taille en octets

6- Date de modification : Date et heure de la dernière modification

7- Nom : Le nom du fichier ou dossier

touch F créer le fichier F

mkdir R créer le répertoire R

mkdir -p R1/R2 créer toute l'arborescence (créer R1 s'il n'existe pas)

mv Source Cible Déplacer le fichier source vers cible

fichier ou chemin vers un fichier | Dossier ou chemin vers un dossier | mv Source1 Source2 Dossier
Les deux seront déplacés dans

rm supprimer les fichiers et les dossiers

rm -r supprimer un dossier et leur contenu

rm -f supprimer un fichier / forcer la suppression

rmdir supprimer des répertoires vides

rmdir -p R1/R2/R3 supprimer toute l'arborescence, il va supprimer R3 puis R2 puis R1

cp Source Cible copier le fichier source vers cible

cp -r Source Cible copier le répertoire source vers cible

* $\Rightarrow$ remplace n caractères ? $\Rightarrow$ remplace 1 caractère \$ $\Rightarrow$ fin
 [0-9] $\Rightarrow$ 0 ou 1 ou 2 ou 3 ou 4 ou 5 ou 6 ou 7 ou 8 ou 9 ^ $\Rightarrow$ début
 [gF] $\Rightarrow$ g ou F (sans espace) [1,7] $\Rightarrow$ 1 et 7 [^_] $\Rightarrow$ tout sauf
 {n} $\Rightarrow$ x n fois

Les critères de la commande find (utilisés pour les fichiers/d répertoire / lien)

find /chemin -name "nom_de_fichier"

rechercher un fichier ou un répertoire par nom exact dans le répertoire /chemin

find . -name "nom_de_fichier"

rechercher un fichier ou un répertoire par nom exact dans le répertoire courant

find /chemin -iname "nom_de_fichier"

rechercher par nom avec insensibilité à la case | -print affiche le chemin de chaque fichier/dossier

find /chemin -type f

recherche uniquement des fichiers | d recherche uniquement des répertoires

find /chemin -size +10M

recherche par taille (fichiers plus grands que 10Mo / plus petit que 1Ko) | -1K

find /chemin -perm G44

recherche les fichiers avec des permissions exactes

find /chemin -user nom_utilisateur

recherche les fichiers appartenant à un utilisateur spécifique

find /chemin -group nom_groupe

recherche les fichiers appartenant à un groupe spécifique

find /chemin -mtime -7

recherche les fichiers modifiés dans les 7 derniers jours

find /chemin -mtime 30

recherche les fichiers modifiés il y a exactement 30 jours

find /chemin -atime -2

recherche les fichiers accessibles dans les 2 derniers jours

find /chemin -mmin -240

recherche les fichiers modifiés il y a moins de 4 heures

find . -newer fichier1.txt

Lister tous les fichiers ayant été modifiés plus récemment que le fichier fichier1.txt (dans le répertoire courant)

find . -iname "..." -delete

supprime tous les fichiers correspondant à un critère

find . -type d -empty

trouver tous les répertoires vides

find /chemin -exec {} -exec A {} \;

Exécute une commande (par exemple A) sur chaque fichier trouvé à l'aide d'un critère (par exemple B)

Combinaison des critères

"et" logique (par défaut)

find /chemin -type f -user utilisateur

un espace = et

"ou" logique (-o)

find /chemin -name "*.jpg" -o -name "*.png"

-o = ou

"non" logique (!)

find /chemin -type f ! -name "*.txt"

negation de

wc /chemin/fichier1

affiche 3 colonnes : nbre de lignes / nbre de mots / nbre de caractères
pour le fichier1

wc -l /chemin/fichier1

affiche le nbre de lignes dans le fichier "fichier1"

wc -w /chemin/fichier1

affiche le nbre de mots dans le fichier "fichier1"

wc -c /chemin/fichier1

affiche le nbre de caractères dans le fichier "fichier1" (compte également les caractères de fin de lignes)

wc -L /chemin/fichier1

affiche la longueur de la ligne la plus longue dans "fichier1"

head /chemin/fichier1

affiche les 10 premières lignes du "fichier1" (par défaut 10)

head -n 7 /chemin/fichier1

affiche les 7 premières lignes du "fichier1"

head -c 7 /chemin/fichier1

affiche les 7 premiers caractères du "fichier1"

tail /chemin/fichier1

affiche les 10 dernières lignes du "fichier1" (par défaut 10)

tail -n 7 /chemin/fichier1

affiche les 7 dernières lignes du "fichier1"

tail -c 7 /chemin/fichier1

affiche les 7 derniers caractères du "fichier1"

sort /chemin/F_n

trie les lignes du fichier F_n par ordre alphabétique croissant
(selon leurs valeurs ASCII)

sort -n /chemin/F_n

trie les lignes de manière inverse

sort -k2 /chemin/F_n

trie le fichier en utilisant la deuxième colonne de chaque ligne
(par défaut, les colonnes sont séparées par des espaces)

- k1, 1 => tri uniquement sur le 1^{er} champ de chaque ligne
- Le premier "1" => on commence à trier à partir du premier champ
- Le second "1" => on arrête de trier après ce premier champ

sort -t ":" -k2 /chemin/F_n

trie en utilisant ":" comme délimiteur

- t: spécifie un délimiteur personnalisé pour les colonnes

sort -u /chemin/F_n

trie les lignes du fichier et affiche uniquement les lignes uniques,
supprimant les doublons

sort -f /chemin/F_n

trie en ignorant la différence entre lettres majuscules et minuscules

sort -b /chemin/F_n

trie sans tenir compte des espaces ou tabulations avant le début des lignes

cut -d ":" -f 1,3 /chemin/F_n (on ne peut utiliser cut sans critères (-d, -f))

extraie la première et la troisième colonne de chaque ligne du fichier "F_n"
où les colonnes sont séparées par des ":"

-d: Définit le délimiteur des champs

cut -f 1 /chemin/F_n

extraie seulement le 1^{er} champ de chaque ligne

cut -f 1,3 /chemin/F_n

affiche les champs 1 et 3

cut -f 2-4 /chemin/F_n

affiche du champ 2 vers champ 4
(2 et 3 et 4)

echo "Hello world"

affiche du texte à l'écran (terminal)

`tr '[:lower:]' '[:upper:]' < /chemin/Fn`

convertir les lignes en minuscules en majuscules

< = redirige le contenu du fichier /chemin/F_n vers la commande tr

`tr '0-9' 'a-j' < /chemin/Fn`

remplacer les chiffres (0-9) par les lettres (a-j) |

`tr -d 'aeiou' < /chemin/Fn`

supprimer toute caractères dans 'aeiou' du fichier F_n

(Le résultat est affiché dans le terminal, mais il n'est pas modifié dans le fichier)
(/chemin/F_n car aucun fichier de sortie n'est redirigé)

`tr '[:lower:]' '[:upper:]' < /chemin/Fn > fichier_sortie.txt`

- Applique la commande tr pour convertir le texte en majuscules.

- redirige le résultat dans le fichier "fichier_sortie.txt"

- si le fichier "fichier_sortie.txt" existe déjà, il sera écrasé avec les nouvelles données.

`tr '[:lower:]' '[:upper:]' < /chemin/Fn >> fichier_sortie.txt`

- Applique la commande tr pour convertir le texte en majuscules

- Ajouter à un fichier sans écraser son contenu existant
à la fin

cat fichier.txt

affiche le contenu de "fichier.txt" dans le terminal

cat fichier1.txt fichier2.txt

affiche d'abord le contenu de "fichier1.txt", suivi de "fichier2.txt"

cat < fichier1.txt > fichier2.txt

copie le contenu de f1 dans f2. Si f2 existe, il sera écrasé

cat < f1.txt >> f2.txt

ajoute le contenu de f1 à la fin de f2

echo "The end" >> f1.txt

ajoute la ligne "The end" à la fin du fichier f1

cat >> f1.txt

"The end"

ctrl+D

} Ajoute la ligne "The end" à la fin du fichier f1

grep "root" /etc/passwd

cherche toutes les lignes contenant le mot root dans le fichier /etc/passwd

grep "mot" fichier1.txt fichier2.txt

cherche le mot "mot" dans plusieurs fichiers

grep -i "root" /etc/passwd

recherche sans tenir compte de la casse (majuscule/minuscule)

grep -v "mot" fichier.txt

-v : négation

affiche uniquement les lignes qui ne contiennent pas "mot"

grep -w "root" /etc/passwd

affiche les mots exacts (ligne qui contient le mot exact) (groot, roots)

ls | grep filter les résultats de ls en utilisant les critères de grep

grep -E: certains caractères spéciaux (comme |, +, (), ...) n'ont plus besoin d'être échappés avec un backslash \, cela rend les motifs plus lisibles

grep -E "go+gle" fichier.txt
grep -E "un(iquement|iversel)" fichier.txt

grep -E "a{3}" fichier.txt

correspond à exactement aaa

avec grep sans -E, il faudrait écrire a\{3\}.

grep -x "Hello world" fichier.txt

correspondance exacte avec une ligne entière (le motif doit matcher toute la ligne)

sed 's/ancien/nouveau' F1.txt

remplace la première occurrence du mot 'ancien' par 'nouveau' dans chaque ligne

sed 's/ancien/nouveau/g' F1.txt

en ajoutant 'g', remplace toutes les occurrences dans chaque ligne

(Le résultat du sed dans ces cas est affiché dans le terminal seulement, mais il n'est pas modifié dans le fichier F1.txt)

sed -i 's/ancien/nouveau/g' F1.txt

Applique les changements directement dans fichier

sed '2d' F1.txt

supprime la 2^{ème} ligne

sed '2,4d' F1.txt

supprime de la ligne 2 à 4

sed '/motif/d' F1.txt

supprime la ligne contient le mot 'motif'

sed '/motif/s/ancien/nouveau/' F1.txt

si une ligne contient 'motif', elle remplacera 'ancien' par 'nouveau' uniquement dans cette ligne

sed 's/^/#/' F1.txt

ajoute un # à chaque début de ligne

sed '\$d' F1.txt

supprime la dernière ligne du fichier

sed 's/ancien/nouveau/3' F1.txt

remplace la troisième (ancien) par (nouveau) | les 3 se trouve dans même ligne
(ligne par ligne non seulement le premier)

sed '/motif/!d' F1.txt

supprime toute les ligne du fichier F1.txt sauf la ligne qui contient 'motif'

sed '/motif/i Texte à insérer' Fichier.txt

insère "Texte à insérer" juste avant chaque ligne contenant le mot 'motif'

sed '1i/Debut' F1.txt

insère "Debut" avant la 1^{ère} ligne du fichier F1.txt

sed '/motif/a Texte à ajouter' F1.txt

ajoute "Texte à ajouter" juste après chaque ligne contenant le mot 'motif'

sed '10a/Fin' F1.txt

ajoute "Fin" après la 10^{ème} ligne du fichier F1.txt

Gestion des utilisateurs et les droits d'accès

Chaque permissions a une valeur numérique

Lecture(r) = 4

Écriture(w) = 2

Exécution(x) = 1

notation symbolique	notation numérique	
---	0	Aucun droit
r--	4	lire
-w-	2	écrire
--x	1	exécuter
rw-	4+2=6	lire + écrire
-wx	2+1=3	écrire + exécuter
r-x	4+1=5	lire + exécuter
rw-x	4+2+1=7	lire + écrire + exécuter

-rwxr-xr--
user(u) group(g) other(o)

fichier

d-rwxr-xr--
a = app

donner

`chmod 644 fichier.txt`

`-rw-r--r--` : lecture et écriture pour le propriétaire ; lecture seule pour le groupe et les autres

`chmod u+r fichier.txt`

Ajoute la permission de lecture pour le propriétaire

`chmod g-w fichier.txt`

Retire la permission d'écriture au groupe

`chmod u+w, o-x fichier.txt`

Ajoute la permission d'écriture pour le propriétaire et retire la permission d'exécution au autre

`chmod u=rwx fichier.txt`

Définit lecture, écriture et exécution pour le propriétaire

(Les permissions par défaut pour un fichier sont 666 (lire+écrire pour tout le monde))
(Les permissions par défaut pour un répertoire sont 777 (lire+écrire+exécuter pour tout le monde))

Umask soustraire des permissions
pour retirer des droits

Valeur Umask	Fichier crée 666	Répertoire crée 777
000	rw-rw-rw- 666	rw-rwxrwx 777
022	rw-r--r-- 644	rw-r-xr-x 755
027	rw-r----- 640	rw-r-xr-- 750
077	rw----- 600	rw-r----- 700

`umask 022`

$666 - 022 = 644 \rightarrow rw-r--r--$ (nouveau droit d'accès par défaut des fichiers qui vont être créés à l'avenir)

`umask 022`

$777 - 022 = 755 \rightarrow rw-r-xr-x$ (nouveau droit d'accès par défaut des répertoires qui vont être créés à l'avenir)